

Fiscal Agent

Architettura, scelte progettuali e integrazioni

Progetto: Assistente fiscale AI — Demo
Aprile 2026

Stack: Python · LlamaIndex · OpenAI · HuggingFace

Data:

Overview

Fiscal Agent è un assistente conversazionale AI che risponde in tempo reale alle domande fiscali dei clienti Fiscozen, personalizzando ogni risposta con i dati specifici del cliente e il contenuto della knowledge base aziendale.

I consulenti e i clienti Fiscozen si trovano spesso a porre domande ricorrenti su regime forfettario, contributi previdenziali, fatturazione e scadenze. Rispondere manualmente a ogni richiesta richiede tempo e comporta il rischio di incoerenze. Il Fiscal Agent risolve questo problema automatizzando le risposte di primo livello, mantenendo il tono di voce Fiscozen e delegando al Customer Success Consultant solo i casi complessi.

Obiettivi specifici



Risposte precise e affidabili

L'agente usa esclusivamente la knowledge base aziendale come fonte. Non inventa dati fiscali.



Personalizzazione per cliente

Ogni risposta tiene conto del regime fiscale, del fatturato e del consulente assegnato al cliente.



Tono di voce coerente

Il sistema prompt applica le linee guida Fiscozen a ogni risposta, senza deroghe.



Escalation intelligente

Quando la domanda è ambigua o rischiosa, l'agente rimanda al Customer Success Consultant assegnato.

Perimetro della demo

La demo opera su un campione di 4 clienti fittizi (Mario Rossi, Giulia Verdi, Luca Martini, Elena Riva) e due documenti fiscali rappresentativi. L'interfaccia è a riga di comando (CLI), con input via terminale o flag.

Scelte progettuali

Il sistema combina quattro componenti distinti — recupero del contesto fiscale (RAG), ricerca web su fiscozen.it, lookup del profilo cliente, generazione del linguaggio — orchestrati in un flusso sequenziale. Ogni scelta è motivata da un requisito preciso.

RETRIEVAL

RAG con LlamaIndex

COSA FA

I documenti fiscali (`.docx`) vengono indicizzati in un *VectorStoreIndex*. L'indice è **persistito su disco** (`storage/`): viene costruito solo al primo avvio e ricaricato nei successivi. Ad ogni domanda, il **retriever** recupera i 4 chunk più semanticamente rilevanti (512 token, overlap 64) e li passa come testo al prompt — senza alcuna chiamata LLM interna da parte di LlamaIndex.

△ PERCHÉ

Il modello LLM non può memorizzare l'intero corpus fiscale nel contesto. Il RAG garantisce che la risposta sia basata su fonti verificabili, riducendo le allucinazioni.

EMBEDDING

HuggingFace locale — BAAI/bge-m3

COSA FA

Gli embedding vettoriali dei documenti e delle query vengono calcolati localmente, senza chiamate API esterne. Il modello scelto è ottimizzato per 100+ lingue, incluso l'italiano, con qualità di retrieval superiore ai modelli multilingue standard.

△ PERCHÉ

Azzera i costi di embedding (nessuna quota OpenAI consumata per il RAG). Il modello multilingue gestisce correttamente termini fiscali italiani senza fine-tuning.

GENERAZIONE

OpenAI GPT — temperatura 0.4

COSA FA

Il modello GPT riceve un prompt strutturato (system + user) e genera la risposta finale in italiano. La temperatura è impostata a 0.4 per bilanciare coerenza e naturalezza del tono di voce.

△ PERCHÉ

Una temperatura nel range 0.3–0.5 è ideale per un assistente: abbastanza stabile da non inventare dati fiscali, abbastanza fluida da risultare naturale nel tono Fiscozen.

WEB SEARCH

Google Custom Search — fallback su fiscozen.it

COSA FA

Se il RAG non trova chunk rilevanti (score < soglia), il sistema estrae le parole chiave fiscali dalla domanda tramite LLM e le usa per cercare su `fiscozen.it` via

△ PERCHÉ

Estendere il contesto al sito pubblico Fiscozen copre guide, servizi e aggiornamenti non presenti nei documenti locali — senza fonti

Google Custom Search API. I top-5 snippet vengono aggiunti al contesto.

esterne. Le keyword sostituiscono la domanda completa per non trasmettere dati personali a Google.

PROMPT DESIGN

Prompt caricati da file — configs/prompts/

COSA FA

Il **system prompt** e il prompt di estrazione keyword sono caricati da file in `configs/prompts/`. Il **user prompt** è costruito dinamicamente con: domanda, dati del cliente, contesto (RAG o web).

△ PERCHÉ

Separare i prompt dal codice permette di modificare tono, regole operative e logica di escalation senza deploy. Ogni file è indipendente e versionabile.

DATI CLIENTE

Excel + Pandas — lookup per nome e cognome

COSA FA

I dati del cliente (regime, cassa, consulente, fatturato, data apertura P.IVA) sono letti da un file `.xlsx` via Pandas. Il DataFrame viene caricato in memoria una sola volta e riutilizzato per ogni query.

△ PERCHÉ

Excel è già il formato nativo usato dai team operativi Fiscozen. Eliminare un layer di database riduce la complessità infrastrutturale nella fase demo, mantenendo un'interfaccia familiare per il team.

DATI CLIENTE

Excel + Pandas — lookup per nome e cognome

COSA FA

I dati del cliente (regime, cassa, consulente, fatturato, data apertura P.IVA) sono letti da un file `.xlsx` via Pandas. Il DataFrame viene caricato in memoria una sola volta e riutilizzato per ogni query.

△ PERCHÉ

Excel è già il formato nativo usato dai team operativi Fiscozen. Eliminare un layer di database riduce la complessità infrastrutturale nella fase demo, mantenendo un'interfaccia familiare per il team.

ESCALATION

Escalation — solo nel codice

COSA FA

Se né il RAG né la web search trovano contesto, `generate_answer()` restituisce direttamente un messaggio di escalation con il nome del consulente assegnato al cliente — senza chiamare l'LLM. Il system prompt non gestisce più questo caso: contiene solo istruzioni per non inventare dati quando il contesto è insufficiente.

△ PERCHÉ

Accentrare l'escalation nel codice è deterministico e non costa chiamate LLM. Evita la duplicazione logica tra codice e prompt che rendeva il comportamento ambiguo.

OSSERVABILITÀ

Logging strutturato — stdlib logging

COSA FA

Ogni passo del flusso emette un log su console con timestamp e livello. I livelli usati: `INFO` per i passi principali (fonte scelta, escalation, caricamento indice), `DEBUG` per i dettagli tecnici (user prompt completo, chunk score, keyword estratte), `WARNING` per configurazione mancante, `ERROR` per errori di rete nella web search. Il logger è configurato in `main.py` e propagato a `utils.py` tramite `getLogger(__name__)`.

⚠ PERCHÉ

Permette allo sviluppatore di vedere esattamente come viene costruito lo user prompt e quale fonte viene usata per ogni risposta, senza modificare il codice. In produzione è sufficiente alzare il livello a `INFO` per ridurre il rumore.

Componenti e dipendenze

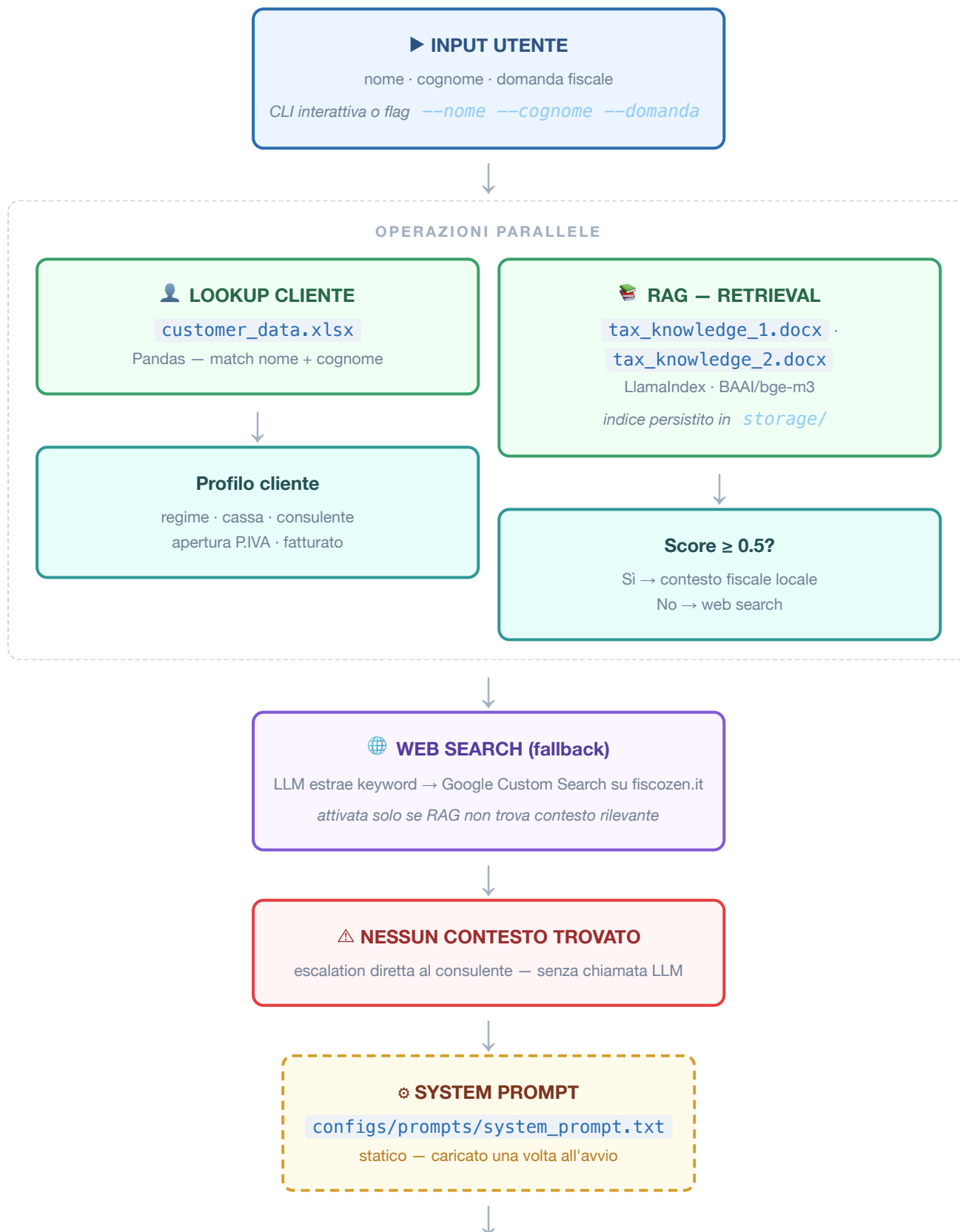
Componente	Libreria / Servizio	Tipo	Ruolo nel sistema
LLM — Generazione	<code>openai</code> <code>>= 1.0</code>	API esterna	Genera la risposta finale in italiano. Riceve il prompt strutturato e restituisce il testo per il cliente.
RAG — Orchestrazione	<code>llama-index-core</code> <code>>= 0.10</code>	Locale	Gestisce indicizzazione vettoriale e retrieval puro. Costruisce il <code>VectorStoreIndex</code> dai documenti e recupera i chunk più rilevanti senza sintetizzare con un LLM interno.
Embedding — Vettorializzazione	<code>llama-index-embeddings-huggingface-sentence-transformers</code>	Locale	Converte testo in vettori numerici. Modello: <code>BAAI/bge-m3</code> . Gira in locale, senza chiamate API. Ottimizzato per retrieval multilingue (100+ lingue, italiano incluso).
Lettura documenti	<code>llama-index-readers-file-docx2txt</code>	File locale	Legge e analizza i file <code>.docx</code> della knowledge base fiscale. Converte il contenuto in nodi testuali per l'indicizzazione.
Dati cliente	<code>pandas</code> <code>>= 2.0</code> <code>openpyxl</code> <code>>= 3.1</code>	File locale	Legge <code>customer_data.xlsx</code> . Esegue il lookup per nome e cognome. Formatta il profilo cliente da inserire nel prompt.
Web search	<code>requests</code> <code>>= 2.31</code> Google Custom Search API	API esterna	Fallback se il RAG non trova contesto rilevante. Cerca su <code>fiscozen.it</code> usando keyword estratte dalla domanda. 100 chiamate/giorno gratuite.
Configurazione ambiente	<code>python-dotenv</code> <code>>= 1.0</code>	Config	Carica le variabili d'ambiente dal file <code>.env</code> locale (chiavi API, modello, temperatura, soglia RAG). Mantiene le credenziali fuori dal codice sorgente.
Logging	<code>logging</code> (stdlib)	Locale	Log strutturati su console con timestamp. <code>DEBUG</code> : user prompt completo, chunk score, keyword estratte. <code>INFO</code> : fonte usata, escalation, caricamento indice. <code>WARNING</code> / <code>ERROR</code> : credenziali mancanti, errori di rete.

Parametri chiave del sistema

Modello LLM	GPT (OpenAI) — via <code>openai</code> SDK
Modello embedding	<code>BAAI/bge-m3</code> — locale, gratuito, ottimizzato per italiano
Chunk size	512 token con overlap di 64 token
Top-k retrieval	4 chunk — i più semanticamente vicini alla domanda
Temperatura LLM	0.4 — bilanciamento coerenza / naturalezza
RAG min score	0.5 — soglia di rilevanza sotto cui si attiva la web search
Web search risultati	5 snippet da fiscozen.it per query
Lingua	Italiano — obbligatorio per system prompt e risposte

Diagramma del flusso

Il diagramma mostra il percorso di ogni richiesta, dall'input dell'utente fino alla risposta generata dal modello.





ASSEMBLAGGIO PROMPT

System: tono di voce + regole | **User:** domanda + profilo cliente + contesto (RAG o web)



OPENAI GPT

temperatura 0.4 · risposta in italiano
escalation al consulente se domanda complessa



✓ RISPOSTA PERSONALIZZATA

tono Fiscozen · dati cliente · [Fonte: knowledge base / fiscozen.it]

 Input /
Output

 Elaborazione
dati

 Configurazione
statica

 Chiamata LLM (API
esterna)

 Output
finale